

AI Copilot Code Review Using Large Language Models

BANDI SRIHITHA

*Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad-500043,
Telangana, India
Roll No: 2420030099*

PRATHIPATI PRANATHI

*Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad-500043,
Telangana, India
Roll No: 2420030043*

EMANI MAHATHI

*Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad-500043,
Telangana, India
Roll No: 2420030093*

Abstract—Software code review is a fundamental software-engineering activity for detecting defects, security concerns, maintainability problems, and violations of coding standards. Manual review, however, is time-consuming and dependent on reviewer expertise. Traditional static analyzers provide deterministic findings but have limited semantic and repository-level awareness, whereas Large Language Models (LLMs) can explain source code and generate review suggestions but may produce generic, inconsistent, or unsupported comments when relevant context is unavailable.

This paper presents an AI Copilot for context-aware automated code review that combines language-specific static analysis, CodeBERT semantic retrieval, FAISS vector search, Retrieval-Augmented Generation (RAG), and the code-specialized CodeLlama model. The proposed framework supports Python, Java, and JavaScript using Pylint, PMD, and ESLint, respectively. Source-code representations are embedded using CodeBERT and indexed in FAISS so that semantically similar code can be retrieved from CodeSearchNet and selected public GitHub repositories.

Static findings, retrieved evidence, source code, and review instructions are assembled into a structured prompt for CodeLlama, which produces an explainable review containing issue descriptions, severity, evidence, suggested fixes, and confidence information. Prototype evaluation reports overall precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56 across the evaluated multilingual static-analysis set. Retrieval evaluation reports Recall@1 of 0.67, Recall@2 of 0.67, Recall@3 of 1.00, and mean reciprocal rank of 0.78. Review-quality evaluation reports precision of 1.00, recall of 0.67, F1-score of 0.80, severity accuracy of 1.00, evidence-grounding rate of 1.00, and unsupported-finding rate of 0.00.

These results demonstrate the feasibility of integrating deterministic analysis, semantic retrieval, and LLM reasoning into a unified explainable code-review workflow, while also highlighting the need for larger matched benchmark evaluations.

Index Terms—automated code review, Large Language Models, CodeLlama, CodeBERT, FAISS, Retrieval-Augmented Generation, static analysis, explainable AI, software engineering

I. INTRODUCTION

Software systems are increasingly developed through rapid, collaborative workflows in which large numbers of source-code changes must be inspected before integration. Code review is therefore a critical software-engineering practice for identifying defects, security weaknesses, maintainability problems, and violations of established development practices. At the same time, manual review requires substantial developer effort and depends on the availability of reviewers with appropriate expertise. The increasing scale of repositories and frequency of code changes make automated assistance

attractive for both individual developers and development teams.

Traditional static-analysis tools provide an established approach to automated review. They apply deterministic rules to source code and can efficiently identify syntax problems, coding-standard violations, suspicious constructs, and selected classes of defects. Their principal limitation is that the findings are generally produced from local program structure and predefined rules. They do not inherently reason over the broader repository history, related implementations, developer intent, or semantically similar code.

Large Language Models (LLMs) provide a complementary capability. Models trained on programming languages can summarize code, explain implementation decisions, identify potential issues, and propose modifications. Code-specialized models such as CodeLlama were introduced specifically to support programming-related tasks [1]. However, LLM-based review can be sensitive to prompt quality and available context. A model may generate a plausible-sounding comment without sufficient evidence, overlook repository-specific conventions, or produce inconsistent severity labels. These limitations motivate systems that combine deterministic evidence with semantic reasoning rather than treating an LLM as the only reviewer.

Recent research has explored several directions. AI-powered code-review agents have investigated the use of LLMs for detecting code smells, potential bugs, and improvement opportunities [7]. CodeAgent uses multiple communicating agents to emulate aspects of collaborative review [8]. RepoAudit combines LLM-driven repository exploration with validation mechanisms to reduce hallucinated findings [9]. Fine-tuned LLM approaches have targeted both review accuracy and comprehensibility [10]. Retrieval-based approaches have also gained attention because relevant historical reviews and contextual evidence can improve generation quality [13], [14].

The proposed work follows a hybrid approach. Instead of choosing between static analysis and LLM reasoning, the framework uses both. Pylint, PMD, and ESLint produce language-specific findings; CodeBERT provides semantic representations of source code; FAISS enables efficient similarity search; RAG supplies relevant retrieved context; and CodeLlama generates the final contextual review.

The main contributions of this work are:

- a unified multilingual code-review pipeline for Python, Java, and JavaScript;

- integration of deterministic static-analysis findings with LLM-generated explanations;
- CodeBERT and FAISS based semantic retrieval for repository-aware context;
- a RAG prompt that combines source code, static findings, retrieved context, and review instructions;
- structured and explainable review output containing severity, evidence, suggested fixes, and confidence; and
- an experimental evaluation covering language-wise static analysis, semantic retrieval, and review quality.

The remainder of this paper presents related work, the research gap, the proposed architecture, datasets and technology stack, methodology, experimental setup, results, comparative analysis, discussion, limitations, and conclusion.

II. RELATED WORK

A. LLM-Based Code Review

The use of LLMs for code review has developed rapidly because these models can process both programming constructs and natural-language instructions. Rasheed *et al.* investigated an LLM-based AI agent for code review and discussed its ability to identify code smells, potential bugs, and improvement opportunities [7]. Such systems offer broader semantic reasoning than conventional rule-based analyzers, but their outputs may still require validation.

CodeAgent introduced a multi-agent architecture for automated code review. It uses a supervisory QA-checker to ensure that agent contributions address the original review question and evaluates tasks including vulnerability introduction, style adherence, inconsistency detection, and code revision [8]. The approach illustrates how decomposition and validation can improve automated review, but multi-agent coordination also introduces additional workflow complexity.

Repository-level auditing introduces a different challenge because an LLM must reason over a large and potentially interconnected codebase. RepoAudit addresses context limitations by allowing an autonomous agent to explore a repository and incorporates validation to reduce false positives [9]. This motivates the present framework’s emphasis on retrieval of relevant code context before generating a review.

B. Pretrained Code Models

CodeBERT is a pretrained model designed to learn representations of programming and natural languages [2]. Its embeddings can be used to represent code semantically and support similarity-based retrieval. CodeXGLUE provides a benchmark collection for code understanding and generation tasks [3], while CodeSearchNet provides resources for semantic code search [4].

CodeLlama provides code-specialized foundation models for code generation and understanding [1]. In the proposed framework, CodeLlama is not used in isolation. Its generation stage receives evidence from static analysis and semantic retrieval, allowing the LLM to operate on a more structured review context.

C. Retrieval-Augmented Code Review

Retrieval-Augmented Generation (RAG) was introduced as a method for combining parametric generation with non-parametric external memory [6]. In software-engineering applications, retrieval can provide historical reviews, related implementations, or repository-specific evidence.

RARe, described as Retrieval-Augmented Reviewer, combines retrieval and generative review by selecting relevant reviews from a codebase and supplying them to a neural generator [13]. LAURA further emphasizes review-knowledge retrieval, context augmentation, and systematic guidance for LLM-based review generation [14]. These approaches support the idea that relevant contextual information can improve the usefulness of automatically generated review comments.

D. Hybrid Static Analysis and LLM Reasoning

Jaoua *et al.* proposed combining large language models with static analyzers through knowledge integration at data preparation, inference, and post-inference stages [12]. Their work explicitly motivates hybrid approaches that combine the precision of rule-based systems with the broader language understanding of LLMs.

The present framework adopts the same general principle but organizes it as an end-to-end application in which static-analysis findings, semantic retrieval, and RAG are integrated before CodeLlama generation. The final output is also structured to include severity and supporting evidence.

E. Review Comprehension and Explainability

Automated review quality is not determined only by whether a system identifies an issue. Developers must also understand why an issue matters and how it can be corrected. Yu *et al.* studied fine-tuning approaches aimed at improving both accuracy and comprehensibility of automated code review [10]. CodeReviewQA provides an assessment framework for code-review comprehension in LLMs [11]. These studies reinforce the importance of understandable review comments and motivate the structured output used in the proposed system.

III. PROBLEM STATEMENT, OBJECTIVES, AND RESEARCH GAP

A. Problem Statement

Software code review is essential for maintaining code quality, but manual inspection is time-consuming and depends on reviewer expertise. Static analyzers reduce this burden by providing deterministic findings, yet they lack broad semantic and repository context. LLMs can provide richer explanations but may produce unsupported or inconsistent recommendations when they are not grounded in relevant code context.

Therefore, there is a need for a hybrid, context-aware automated code-review framework that combines deterministic analysis, semantic retrieval, retrieval-augmented prompting, and code-specialized LLM reasoning.

TABLE I
RESEARCH GAPS AND PROPOSED RESPONSES

Observed Gap	Response in Proposed Framework
Limited repository context	CodeBERT embeddings and FAISS retrieval
Static and LLM review separated	Static findings are included in LLM prompt
Generated comments difficult to validate	Structured output with evidence and confidence
Inconsistent severity labels	Deterministic severity normalization
Limited language coverage	Python, Java, and JavaScript analyzers
Inconsistent result presentation	Streamlit structured review report

B. Objectives

The project objectives are:

- 1) develop multi-language code analysis for Python, Java, and JavaScript;
- 2) integrate language-specific static analyzers to detect deterministic rule-based issues;
- 3) generate semantic code embeddings using CodeBERT;
- 4) build a FAISS-based vector database for efficient similar-code search;
- 5) implement a RAG pipeline using CodeSearchNet and selected GitHub repositories;
- 6) construct structured prompts containing source code, static findings, retrieved context, and review instructions;
- 7) use CodeLlama to generate contextual reviews and suggested fixes;
- 8) provide explainable severity-ranked recommendations with supporting evidence and confidence information; and
- 9) evaluate retrieval and review performance using appropriate benchmark and repository data.

C. Research Gap

Several limitations can be observed in existing approaches. First, existing systems can have limited repository or contextual awareness. Second, static analysis and LLM review are often treated as separate stages rather than complementary evidence sources. Third, generated comments can be difficult to validate. Fourth, severity labels can vary between generated outputs. Finally, many approaches do not provide a unified multilingual review interface.

D. Innovation, Novelty, and Expected Outcomes

The innovation of the proposed framework is primarily architectural. It combines repository-aware FAISS-based RAG, CodeLlama, static analysis, intelligent retrieval of similar code, explainable suggestions, and severity-prioritized reporting in one application.

The documented novelty includes CodeBERT-based semantic code retrieval, repository-aware contextual generation, multi-language support, and structured review output.

Expected outcomes include:

- automated review for Python, Java, and JavaScript;
- improved context-aware code analysis;
- explainable review suggestions;
- severity-based prioritization;
- reduced manual review effort;
- improved developer productivity and maintainability; and
- a scalable basis for future programming-language support.

IV. PROPOSED ARCHITECTURE

The proposed framework is a hybrid context-aware code-review system. Its workflow follows eight major stages:

- 1) source-code upload;
- 2) programming-language detection;
- 3) language-specific static analysis;
- 4) CodeBERT embedding generation;
- 5) FAISS similarity retrieval;
- 6) contextual prompt construction;
- 7) CodeLlama review generation; and
- 8) explainable review reporting.

A. Overall Architecture

Fig. 1 illustrates the complete architecture. The first stage receives source code and identifies the programming language. Language-specific analyzers then generate deterministic findings. In parallel, source-code representations are embedded using CodeBERT and used to retrieve similar code from the indexed corpus.

The retrieved evidence and static-analysis findings are merged with the original source code and review instructions. This contextual prompt is sent to CodeLlama. The final response is parsed and presented as an explainable report containing the identified issue, explanation, severity, suggested fix, supporting evidence, and confidence estimate.

B. Static Analysis Layer

The static-analysis layer is responsible for deterministic and language-specific inspection. Python source code is routed to Pylint, Java source code to PMD, and JavaScript source code to ESLint.

The analyzers provide rule-based findings that form an explicit evidence layer for subsequent reasoning.

This stage is important because an LLM-only architecture may generate plausible but unsupported comments. Static findings provide concrete signals that can be retained and referenced in the generated review. The design therefore treats static analysis as an evidence generator rather than as a replacement for semantic reasoning.

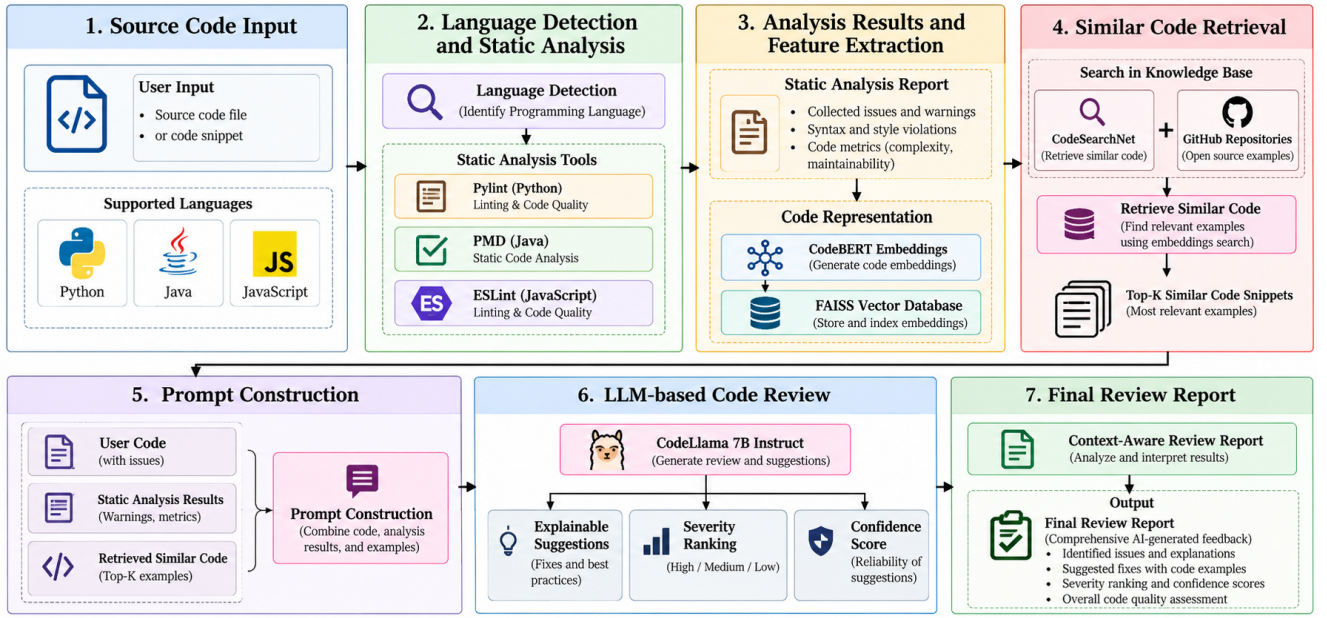


Fig. 1. Overall architecture of the proposed AI-powered code analysis and review system.

Fig. 1. Proposed hybrid context-aware architecture for AI-assisted code review.

C. Semantic Retrieval Layer

The semantic retrieval stage addresses the lack of repository context. Code chunks are represented using CodeBERT embeddings and indexed in FAISS. The system retrieves semantically similar chunks rather than relying only on lexical matching.

The retrieval corpus is based on CodeSearchNet and selected public GitHub repositories. The prototype retrieval corpus contains eight chunks. This is sufficient for feasibility testing but should not be interpreted as a large-scale repository benchmark.

D. Retrieval-Augmented Generation

RAG connects retrieval with generation. For a submitted code fragment, the system constructs a contextual prompt from four main information sources:

- 1) original source code;
- 2) static-analysis findings;
- 3) retrieved semantic context; and
- 4) review instructions.

This architecture differs from a direct “source code to LLM” workflow. Retrieved evidence provides additional context, while static findings provide deterministic signals. The LLM is used for explanation, reasoning, prioritization, and suggested remediation.

E. CodeLlama Generation

CodeLlama is used as the code-specialized generation component. The model receives a structured prompt and produces review information.

The proposed implementation uses a pretrained code-specialized LLM together with retrieval and prompt construction rather than relying on task-specific model fine-tuning.

F. Explainable Output

The final review is structured into fields including:

- issue title;
- issue description;
- severity;
- evidence;
- confidence; and
- suggested fix.

This output format is intended to make the review easier for developers to inspect and to provide a consistent presentation across programming languages.

G. How the Datasets Fit the Workflow

The datasets are used at different stages rather than as one homogeneous training set.

CodeSearchNet supplies source-code material for semantic retrieval. CodeXGLUE provides benchmark resources for code understanding and review-related evaluation. Selected GitHub public repositories provide repository context for retrieval and testing.

V. METHODOLOGY

A. End-to-End Workflow

The implementation follows eight stages, shown in Fig. 2.

- 1) **Source-code upload:** the user submits the code to the application.
- 2) **Language detection:** the application determines whether the input is Python, Java, or JavaScript.

TABLE III
TECHNOLOGY STACK AND FUNCTIONAL ROLES

Component	Function
Python	Application implementation
Streamlit	User interface and review presentation
CodeLlama	Code review generation and explanation
CodeBERT	Semantic code representation
FAISS	Vector similarity search
Pylint	Python static analysis
PMD	Java static analysis
ESLint	JavaScript static analysis
RAG	Context retrieval and prompt augmentation

B. Technology Stack

The implementation uses Python as the primary programming language and Streamlit as the user interface.

CodeLlama is the code-oriented LLM, CodeBERT is used for embeddings, and FAISS is used for vector similarity search.

Pylint, PMD, and ESLint provide language-specific static analysis.

C. Feasibility

The project identifies technical, economic, operational, and time feasibility.

Technically, the framework uses open-source AI models and libraries and can integrate language-specific static-analysis tools.

Economically, it avoids expensive model training and primarily uses open-source technologies.

Operationally, the Streamlit interface is intended to provide a user-friendly workflow, while the modular architecture permits future expansion.

The project is based on existing AI and RAG technologies and can be implemented through multiple development phases.

D. Implementation Plan

The implementation proceeds from problem and literature analysis to architecture design, integration of the static-analysis and retrieval components, LLM prompt construction, interface development, testing, and evaluation.

This phased implementation supports the modular architecture and permits individual components to be evaluated before end-to-end integration.

VII. EXPERIMENTAL SETUP

The evaluation intentionally distinguishes system behavior from claims about universal code-review correctness.

Precision, recall, F1-score, and accuracy are reported for the supplied prototype evaluation.

The prototype evaluation uses a multilingual static-analysis dataset and a small retrieval corpus. The experimental design measures the performance of the system at multiple levels rather than relying on a single aggregate score.

TABLE IV
STATIC-ANALYSIS PERFORMANCE BY PROGRAMMING LANGUAGE

Language	Precision	Recall	F1	Accuracy
Python	0.50	1.00	0.67	0.50
Java	0.25	1.00	0.40	0.25
JavaScript	1.00	1.00	1.00	1.00
Overall	0.56	1.00	0.71	0.56

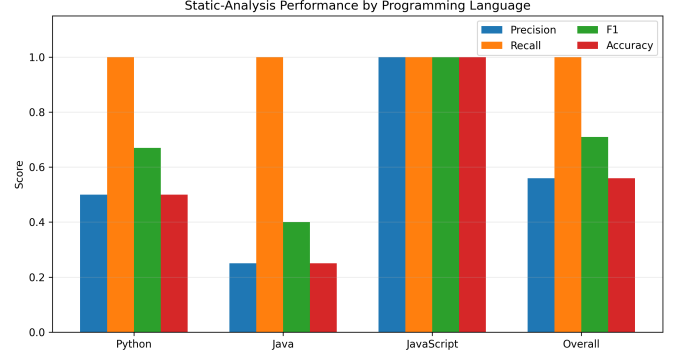


Fig. 3. Static-analysis performance of the proposed project by programming language.

First, the static-analysis stage is evaluated separately for Python, Java, and JavaScript.

Second, the semantic retrieval component is evaluated using rank-based retrieval metrics.

Third, the final review output is evaluated for issue identification and structured review characteristics.

The experiments are intended as a feasibility evaluation of the integrated prototype. They should not be interpreted as a matched comparison against every published code-review model.

VIII. RESULTS

A. Static-Analysis Performance

Table IV reports the measured performance for each programming language.

Python obtained precision of 0.50, recall of 1.00, F1-score of 0.67, and accuracy of 0.50.

Java obtained precision of 0.25, recall of 1.00, F1-score of 0.40, and accuracy of 0.25.

JavaScript obtained 1.00 across all four reported metrics.

The overall prototype values were precision 0.56, recall 1.00, F1-score 0.71, and accuracy 0.56.

The language-wise differences are important.

The perfect recall reported for the prototype indicates that all evaluated static findings were captured by the evaluation procedure.

The lower precision in Python and Java indicates that additional findings were produced.

These results support the need for contextual reasoning and evidence validation rather than interpreting every static or generated finding as an equally reliable defect.

TABLE V
SEMANTIC RETRIEVAL RESULTS

Metric	Result
Recall@1	0.67
Recall@2	0.67
Recall@3	1.00
MRR	0.78

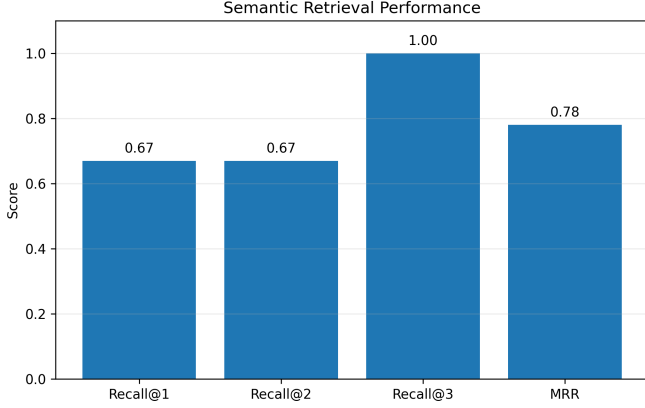


Fig. 4. Semantic retrieval performance of the CodeBERT-FAISS component.

B. Retrieval Performance

The retrieval component achieved Recall@1 of 0.67 and Recall@2 of 0.67.

Recall@3 reached 1.00, meaning that every evaluated retrieval case had a relevant result within the top three positions.

The reported MRR of 0.78 indicates that relevant results tended to occur relatively high in the ranked list.

The results demonstrate feasibility on the prototype retrieval corpus.

However, the corpus is small, and retrieval performance may change substantially when the system is evaluated against larger repositories containing different programming styles, project structures, and domain-specific code.

C. Review Quality

Table VI summarizes the reported review-quality results. Precision is 1.00, recall is 0.67, and F1-score is 0.80.

Severity accuracy and evidence-grounding rate are both reported as 1.00, while the unsupported-finding rate is 0.00.

These values describe the evaluated prototype review behavior and should not be interpreted as universal LLM accuracy.

In particular, the project’s evaluation is based on a small selected set and does not replace human assessment of the usefulness and correctness of generated review comments.

IX. COMPARATIVE ANALYSIS

The comparative analysis separates qualitative system capabilities from numerical evaluation.

This distinction is necessary because the reviewed approaches use different datasets, tasks, labels, and scoring procedures.

TABLE VI
REVIEW QUALITY EVALUATION

Metric	Result
Precision	1.00
Recall	0.67
F1-score	0.80
Severity accuracy	1.00
Evidence grounding rate	1.00
Unsupported finding rate	0.00

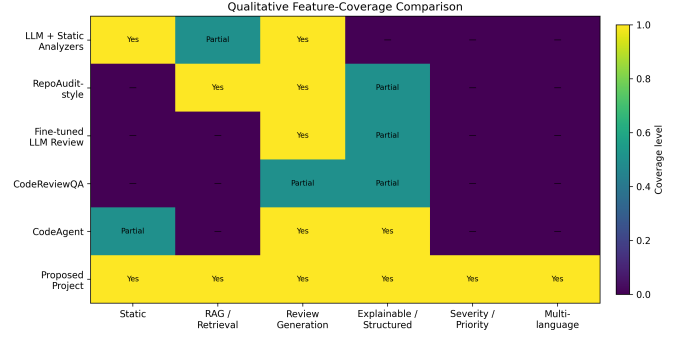


Fig. 5. Qualitative feature-coverage comparison of selected approaches and the proposed project.

A. Qualitative Comparison

Table VII compares the documented scope of selected approaches.

“Not stated” means that the supplied comparison material did not provide sufficient evidence to mark a feature as implemented.

The proposed project combines six functional areas:

- 1) language-specific static analysis;
- 2) semantic retrieval;
- 3) review generation;
- 4) structured and explainable output;
- 5) severity mapping; and
- 6) multilingual support.

This integrated scope is the principal system-level contribution rather than a claim of a new standalone learning algorithm.

B. Numerical Comparison

Table VIII reports the numerical values supplied in the comparative-analysis document.

The proposed project records higher recall than the reported comparison values, but the proposed precision and accuracy are not directly comparable because the tasks differ.

The CodeReviewer and baseline values concern code-change quality estimation, whereas the proposed project’s values concern static-analysis findings on a small multilingual prototype dataset.

Therefore, Fig. 6 is a reported-metrics reference rather than evidence of universal superiority.

TABLE VII
QUALITATIVE COMPARISON OF CODE-REVIEW APPROACHES

Approach	Static	RAG	Review Gen.	Structured Output	Severity	Multi-lang.	Main Difference
LLM + Static Analyzers (2025)	Yes	Partial	Yes	Not stated	Not stated	Not stated	Combines analyzer signals and LLM generation.
RepoAudit-style System	Not stated	Yes	Yes	Partial	Not stated	Not stated	Repository-level auditing and validation are central.
Fine-tuned LLM Review	Not stated	Not stated	Yes	Partial	Not stated	Not stated	Requires task-specific review data and training.
CodeReviewQA	Not central	Not central	Evaluation	Evaluation	Not central	Not central	Primarily a review-comprehension benchmark.
CodeAgent (2024)	Partial	Not central	Yes	Yes	Not stated	Not stated	Multi-agent coordination increases workflow complexity.
Proposed Project	Yes	Yes	Yes	Yes	Yes	Yes	Integrates all listed functions in one application.

TABLE VIII
REPORTED METRICS ACROSS DIFFERENT TASKS AND DATASETS

Model/System	Task	Precision	Recall	F1	Accuracy
Proposed Project	Small multilingual static-analysis dataset	0.560	1.000	0.710	0.560
CodeReviewer	Code-change quality estimation	0.786	0.656	0.715	0.739
T5 baseline	Code-change quality estimation	0.708	0.572	0.633	0.668
CodeT5 baseline	Code-change quality estimation	0.704	0.590	0.642	0.671
Transformer baseline	Code-change quality estimation	0.745	0.461	0.569	0.652

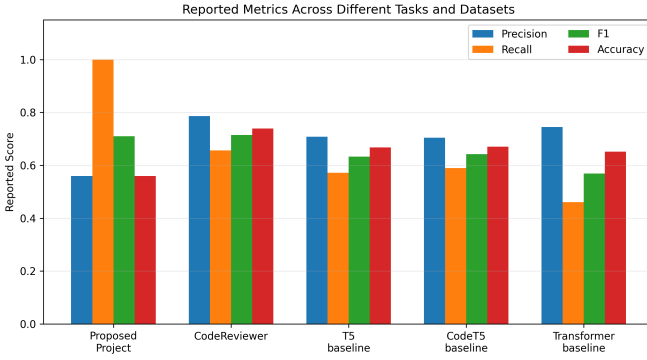


Fig. 6. Reported metric comparison across systems evaluated under different tasks and datasets.

X. DISCUSSION

The experimental results demonstrate that the proposed pipeline is operational across its principal stages.

The static-analysis results confirm that the language-specific analysis layer can be applied to three programming languages in a common application.

The retrieval results demonstrate that semantic search can supply relevant context for the evaluated prototype cases.

The review-quality results indicate that the structured output stage can produce consistent severity and evidence fields under the selected evaluation procedure.

The most important architectural advantage is the separation of evidence generation from reasoning.

Static analyzers provide deterministic signals, retrieval supplies contextual information, and the LLM performs explanation and recommendation.

This separation makes it possible to inspect intermediate evidence rather than treating the LLM response as an opaque decision.

The retrieval stage also introduces an important scalability consideration.

As repository size increases, the quality of the retrieved context becomes increasingly important. Poorly retrieved code can introduce irrelevant information into the prompt and potentially affect the generated review.

Consequently, future work should investigate retrieval thresholds, chunking strategies, metadata filtering, and larger repository corpora.

The severity-normalization layer provides another practical benefit. LLMs can use different terminology or severity scales in different responses.

Mapping results to a consistent set of severity categories makes review reports easier to interpret.

However, severity accuracy in the current evaluation should not be treated as proof that the system’s severity labels correspond perfectly to developer judgments; human evaluation is required for that claim.

The system also has a practical developer-facing advantage through the Streamlit interface.

The interface provides a single workflow for submitting code and viewing the structured review.

This supports the project’s stated objective of reducing manual review effort and providing developer-friendly explanations.

XI. INNOVATION, CREATIVITY, AND FEASIBILITY

A. Innovation

The innovation of the framework lies primarily in integration.

The system combines repository-aware retrieval, static analysis, CodeLlama generation, and explainable output instead of relying on a single model or analyzer.

The retrieval stage attempts to provide similar code context before review generation, while the static-analysis layer contributes deterministic findings.

B. Creativity

The framework simulates an expert-review workflow in which deterministic tools identify candidate issues and a code-specialized LLM explains them using contextual evidence.

The modular architecture allows individual components to be replaced or expanded without redesigning the complete application.

For example, additional static analyzers or programming languages can be integrated in future versions.

C. Technical Feasibility

The system uses established open-source models, libraries, and analysis tools.

CodeBERT and FAISS provide an established route for semantic retrieval, while CodeLlama supplies code-oriented language generation.

Pylint, PMD, and ESLint are compatible with language-specific static analysis.

These components can therefore be integrated within a Python application.

D. Economic and Operational Feasibility

The framework relies primarily on open-source technologies and does not require expensive model training.

The Streamlit interface provides a relatively simple interaction model for developers.

The modular architecture also supports future integration with developer workflows, IDEs, and CI/CD systems.

XII. LIMITATIONS AND FUTURE WORK

The current evaluation has several limitations.

First, the prototype dataset is small.

Second, the retrieval corpus is limited, so the reported retrieval metrics should not be generalized to large repositories.

Third, the numerical comparison uses different tasks and datasets and therefore cannot establish a direct ranking against published systems.

Fourth, human evaluation of the generated comments has not been established in the supplied experimental results.

The absence of matched benchmark experiments is particularly important.

A stronger evaluation would use the same dataset, train/test split, label definition, evaluation metrics, and preprocessing procedure for the proposed system and selected baselines.

This would allow a statistically meaningful comparison.

Future work should therefore include:

- larger and more diverse repositories;
- human evaluation of review usefulness;
- ablation studies;

- latency and memory measurements;
- matched benchmark experiments;
- additional programming languages;
- IDE integrations; and
- CI/CD integrations.

Ablation studies should separately remove static analysis, retrieval, RAG, and structured output to determine the contribution of each component.

A further direction is improved retrieval.

The current system can be extended with repository meta-data, file-path awareness, symbol-level indexing, dependency relationships, and historical review information.

These additions could help retrieve context that is more closely related to the code under review.

Another direction is evaluation of generated fixes.

The current results emphasize review findings and structured explanations.

Future experiments could assess whether suggested fixes compile, pass tests, and preserve intended functionality.

Such an evaluation would provide a stronger measure of practical usefulness than text similarity alone.

XIII. CONCLUSION

This paper presented an AI Copilot for context-aware automated code review that combines language-specific static analysis, CodeBERT semantic retrieval, FAISS vector search, Retrieval-Augmented Generation, CodeLlama, and structured explainable reporting.

The system supports Python, Java, and JavaScript and integrates deterministic analysis with contextual LLM reasoning.

The reported prototype results show overall static-analysis precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56.

The retrieval component achieved Recall@3 of 1.00 and MRR of 0.78.

Review-quality evaluation reported precision of 1.00, recall of 0.67, F1-score of 0.80, severity accuracy of 1.00, evidence-grounding rate of 1.00, and unsupported-finding rate of 0.00.

These results demonstrate the feasibility of the integrated architecture on the selected prototype evaluation set.

The primary contribution is the combination of complementary mechanisms into one developer-oriented workflow:

- static analyzers supply deterministic evidence;
- CodeBERT and FAISS supply semantic context;
- RAG organizes the evidence; and
- CodeLlama provides explanation and suggested remediation.

The results should nevertheless be interpreted conservatively.

The prototype dataset and retrieval corpus are limited, and the reported comparison with published systems uses different tasks and datasets.

Larger matched evaluations, human assessment, ablation studies, and performance profiling are required before broader claims can be made.

With these extensions, the framework can evolve toward a more comprehensive context-aware code-review assistant for practical software-development environments.

REFERENCES

- [1] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code,” arXiv preprint arXiv:2308.12950, 2023.
- [2] Z. Feng *et al.*, “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [3] S. Lu *et al.*, “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [4] H. Husain *et al.*, “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search,” in *Proc. 2020 Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019.
- [5] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [6] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [7] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, “AI-powered Code Review with LLMs: Early Results,” arXiv preprint arXiv:2404.18496, 2024.
- [8] X. Tang *et al.*, “CodeAgent: Autonomous Communicative Agents for Code Review,” in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing*, pp. 11279–11313, 2024.
- [9] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, “RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing,” in *Proc. 42nd Int. Conf. Machine Learning*, PMLR, vol. 267, pp. 21083–21100, 2025.
- [10] Y. Yu, G. Rong, H. Shen, H. Zhang, D. Shao, M. Wang, Z. Wei, Y. Xu, and J. Wang, “Fine-Tuning Large Language Models to Improve Accuracy and Comprehensibility of Automated Code Review,” *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 1, pp. 1–26, 2025.
- [11] H. Y. Lin, C. Liu, H. Gao, P. Thongtanunam, and C. Treude, “CodeReviewQA: The Code Review Comprehension Assessment for Large Language Models,” in *Findings of ACL 2025*, pp. 9138–9166, 2025.
- [12] I. Jaoua, O. Ben Sghaier, and H. Sahraoui, “Combining Large Language Models with Static Analyzers for Code Review Generation,” in *Proc. 2025 IEEE/ACM 22nd Int. Conf. Mining Software Repositories*, 2025, doi: 10.1109/MSR66628.2025.00038.
- [13] Q. Meng, X. Zhang, Z. Ren, and J. Visser, “Code Review Automation using Retrieval Augmented Generation,” arXiv preprint arXiv:2511.05302, 2025.
- [14] Y. Zhang, Y. Zhang, Z. Sun, Y. Jiang, and H. Liu, “LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM,” in *Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering*, pp. 2983–2995, 2025.
- [15] “An Insight into Security Code Review with LLMs: Capabilities, Obstacles, and Influential Factors,” 2024.
- [16] “Evaluating Source Code Quality with Large Language Models: A Comparative Study,” 2024.
- [17] “Automating Code Review Activities by Large-Scale Pre-training,” 2022.
- [18] “Automated Code Review Using Large Language Models with Symbolic Reasoning,” 2025.